

BM4152 Biosignal Processing

Assignment 1

- This is an individual assignment.
 - Use the data files (.npz) that are attached as instructed throughout the assignment.
 - Submission guidelines:
 - Report: Upload a PDF to Moodle. Include observations and discussions with relevant plots to support your answers.
 - Python scripts / notebooks: Upload a single ZIP file with all .py files (or a single .ipynb notebook) to Moodle. Name each script/section according to the question number. Include comments for readability.
-

Introduction

This assignment involves implementing digital filters in Python and applying them to biomedical signals. You are expected to generate and analyse the results and become familiar with the practical limitations of implemented digital filters.

1 Smoothing Filters

1.1 Moving Average MA(N) Filter

A MA(N) filter can be visualised as a moving window across the signal. A point of the filtered signal $y(n)$ is derived from the average of the windowed points on the input signal $x(n)$

$$y(n) = \frac{1}{N} \sum_{k=0}^{N-1} x(n-k) \quad (1)$$

1.1.1 Preliminaries

1. Load `ECG_template.npz`.
Acquisition parameters of this signal are:
Sampling frequency - 500 Hz
Amplitude units - mV
2. Plot the loaded signal with the adjusted time scale (according to f_s). Observe typical ECG characteristics.
3. Add white Gaussian noise of 5 dB.
Scale the zero-mean Gaussian noise so that the resulting SNR (computed from the measured signal power) equals the target value in dB. Name the noisy signal `nECG`.
4. Plot the power spectral density (PSD) estimate of `nECG` using
`scipy.signal.periodogram(x, fs=fs, window=..., nfft=...)`

1.1.2 MA(3) filter implementation with a custom function

1. Write a Python function for an MA(3) filter using Equation (1). (Do not use a built-in filtering function for this part).
Name the filtered signal `ma3ECG_1`.
2. Derive the group delay.
3. Plot the delay-compensated `ma3ECG_1`, `ECG_template` and `nECG` on the same plot and compare.
4. Produce overlapping PSDs of `ma3ECG_1` and `nECG` and compare the filtering effect.

1.1.3 MA(3) filter implementation with a built-in function

1. Use `scipy.signal.lfilter(b, a, x)` to filter `nECG` with an MA(3) filter while compensating for the group delay.
Name the filtered signal `ma3ECG_2`.
2. Plot `nECG`, `ECG_template` and `ma3ECG_2` on the same plot and compare.
3. Inspect the magnitude response, phase response and pole-zero plot of the MA(3) filter using `scipy.signal.freqz(b, a)` for magnitude/phase, and `numpy.roots()` for the pole-zero plot.

1.1.4 MA(10) filter implementation with a built-in function

1. Identify the improvement of the MA(10) filter over the MA(3) filter by using `scipy.signal.freqz(b, a)` for magnitude/phase, and `numpy.roots()` for the pole-zero plot.
2. Filter `nECG` signal using the above MA(10) filter while compensating for the group delay.
Name the filtered signal `ma10ECG`.
3. Plot `nECG`, `ECG_template`, `ma3ECG_2` and `ma10ECG` on the same plot to compare the improvement.

1.1.5 Optimum MA(N) Filter Order

1. Write a Python function to calculate the mean-squared-error (MSE) between the noise-free signal and the filtered signal for an MA(N) filter.
2. Hence test for a range of N values and determine the optimum filter order which gives the minimum MSE by plotting MSE vs. N.
3. Suggest reason/s for large MSE values at low and high filter orders.

1.2 Savitzky-Golay SG(N,L) Filter

The Savitzky-Golay filter fits a polynomial of order N to an odd number of data points $L' = 2L + 1$ (where L' is an odd integer) in a predefined window in a least-squares sense. A unique solution requires $N \leq L' - 1$.

1.2.1 Application of SG(N,L) Filter

1. Apply an SG(3,11) filter on `nECG` using `scipy.signal.savgol_filter(x, window_length, polyorder)`.
Name the filtered signal `sg311ECG`.
2. Plot `nECG`, `ECG_template` and `sg311ECG` on the same plot and compare.

1.2.2 Optimum SG(N,L) filter parameters

1. Calculate the MSE values for a range of parameters of the SG(N,L) filter and determine the optimum filter parameters which gives the minimum MSE. (Hint: use matplotlib's `pcolormesh()` or a 3-D `plot_surface()`)
2. Plot `ECG_template`, `sg311ECG` and the signal obtained from the optimum SG(N,L) filter on the same plot to compare the improvement.
3. Compare the signal features and computational complexity of the optimum filtered signals derived from the MA(N) and SG(N,L) filters.

2 Ensemble Averaging

In the case of overlapping signal and noise spectra, synchronised averaging is an effective and simple method for noise removal with minimal signal distortion. However, synchronised averaging requires either multiple measurements (e.g. evoked potentials) or a single signal with repetitive patterns (e.g. ECG).

2.1 Signal with multiple measurements

Consider the signal with multiple measurements to be the **Auditory Brainstem Response (ABR)** which is the early part of the auditory evoked potential. The ABR is generated as a response to an auditory stimulus and recorded from EEG electrodes placed on the scalp. Multiple ABRs are recorded by multiple stimulations given at a constant rate. The amplitude of the ABR is in the range of μV which is over-shadowed by the ongoing EEG noise which is in the range of mV. This section uses a recorded ABR pulse train (over 1000 ABRs) having properties given in Table 1 and removes underlying EEG noise using ensemble averaging.

Table 1: Signal Properties

Name of recording	ABR_rec.npz
Sampling frequency	40 kHz
Amplitude units	mV
Duration of ABR	10 ms from the point of stimulation
ABR_rec column 1	auditory pulse stimulus (0.1 ms, $f_s = 21.1$ Hz)
ABR_rec column 2	ABR + EEG

2.1.1 Preliminaries

1. Load `ABR_rec.npz`.
2. Plot the train of stimuli and ABRs on a single plot and observe amplitudes and waveforms.
3. Determine a voltage threshold (e.g. 50) to automatically detect the likely stimuli occurrences, using boolean indexing (`numpy.where()`).
4. Extract the actual stimulus points (the leading edge of each detected pulse).
5. Window ABR epochs according to the extracted stimulus points. Use a window length of 12 ms (-2 ms to 10 ms) relative to the stimulus time point (i.e. -80 to 399 sample points at $f_s = 40$ kHz).
6. Calculate the ensemble average of all extracted epochs.
7. Plot the ensemble-averaged ABR waveform. Your plot should resemble Figure 1 (labelling aside).

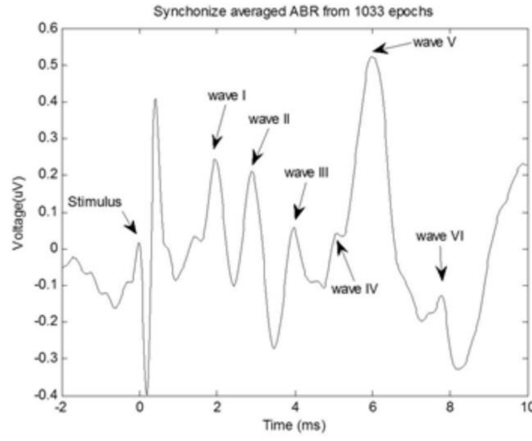


Figure 1: Auditory Brainstem Reponse (ABR)

2.1.2 Improvement of the SNR

Using the ensemble-averaged ABR (`ensmb1_avg`) as the template, write a Python function to calculate an array containing progressive MSEs as per the following equation:

$$MSE_k = \sqrt{\frac{\sum_{n=1}^N (y(n) - \hat{y}_k(n))^2}{N}}, \quad k = 1, 2, \dots, M \quad (2)$$

Where, $y(n)$ is the template, N is the length of a single ABR epoch, M is the number of epochs in the ensemble average, and $\hat{y}_k(n)$ is the progressive average of epochs defined as

$$\hat{y}_k(n) = \frac{1}{k} \sum_{i=1}^k x_i(n) \quad (3)$$

1. Plot a graph of MSE_k against k and describe the behaviour with reference to the theoretically derived improvement in SNR.

2.2 Signal with repetitive patterns

Consider an almost periodic recording of an ECG pulse train having acquisition parameters given in Table 2. The task in this section is to add Gaussian white noise to this signal to emulate practical conditions and extract a single denoised ECG pulse using ensemble averaging. Here, the use of ensemble averaging is justified since the ECG spectrum inevitably overlaps with white noise.

Table 2: Signal Properties

Name of recording	ECG_rec.npy
Recorded Lead	II
Sampling frequency	128 Hz
Amplitude units	mV

2.2.1 Viewing the signal and addition of Gaussian white noise

1. Load `ECG_rec.npy`.
2. Plot the data and observe amplitudes and waveforms. The scale and labeling should be like Figure 2.
3. Extract a single PQRST waveform and define it as `ECG_template`.
4. Add Gaussian white noise of 5 dB to `ECG_rec` and name it `nECG`.

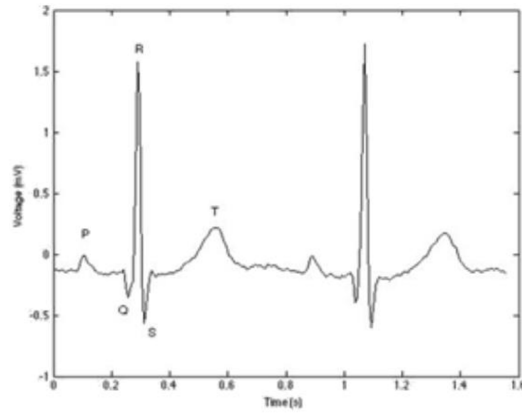


Figure 2: ECG signal

2.2.2 Segmenting ECG into separate epochs and ensemble averaging

1. Calculate the normalised cross-correlation between `ECG_template` and `nECG`. This will be used to segment the ECG wave into epochs.
2. Plot the normalised cross-correlation values against the adjusted lag axis, converted to a time axis.
3. Segment ECG pulses by defining a threshold for the cross-correlation and storing them in a separate array/matrix.
4. Calculate and plot the improvement in SNR as the number of ECG pulses included in the ensemble average increases.
5. Plot and compare (on one graph) a selected noisy ECG pulse and two arbitrarily-selected ensemble-averaged ECG pulses.
6. Suggest a justification to the claim ‘it is a better method to use points of maximum correlation with noisy ECG pulse train rather than merely detecting the R-wave to segment the ECG pulse train into separate epochs’.

3 Designing FIR Filters Using the Windowing Method

3.1 Characteristics of window functions

For the following questions, use `scipy.signal.firwin()` to build filter taps, `scipy.signal.freqz()` to inspect their frequency response, and `scipy.signal.get_window()` to generate window functions directly.

1. The rectangular window is the simplest window function. Explain the effect of the length of this window (M) using overlaying plots of the impulse response for $M = 5, 50, 100$.
2. For a lowpass filter with cutoff frequency $\omega_c = 0.4\pi$ (normalised frequency), plot the comparative magnitude response with a linear magnitude scale and their comparative phase response plot for $M = 5, 50, 100$. (for example, refer the slide ‘Rectangular window function’)
3. To improve the magnitude response, smoother windows have been designed removing discontinuities of the rectangular window. Plot and explain the following comparative characteristics of the rectangular, Hanning, Hamming and Blackman windows of length $M = 50$.
 - Morphology of the window
 - Magnitude response with a linear magnitude scale

- Magnitude response with a logarithmic magnitude scale
- Phase response

3.2 FIR filter design and application using the Kaiser window

The Kaiser window is a generic function which can approximate a variety of windows by varying the shaping parameter β and the window length M . In this section, a lowpass and a highpass filter using the Kaiser window and a FIR comb filter should be designed to filter out noise embedded in an ECG signal of which the information is given in Table 3.

Table 3: Signal Properties

Name of recording	ECG_with_noise.npz
Recorded Lead	II
Sampling frequency	500 Hz
Amplitude units	mV

1. Plot the time domain signal and the power spectral density (PSD) estimate of the above signal using `scipy.signal.periodogram`. Identify its frequency content with respect to a bandwidth of a normal ECG.
2. Decide on parameters for the filters below to apply to the ECG. For a guide, use a highpass cutoff frequency of 5 Hz and a lowpass cutoff frequency of 125 Hz.
 - (a) Highpass filter: f_{pass} , f_{stop} , δ .
 - (b) Lowpass filter: f_{pass} , f_{stop} , δ .
 - (c) Notch filter: f_{stop1} , f_{stop2} , f_{stop3} .
3. Calculate the relevant β and M values for the highpass and lowpass filters.
4. Visualize the windows (highpass and lowpass), magnitude responses, and phase responses of the above filters.
5. Apply these filters one by one to the `ECG_with_noise.npy` signal with the necessary compensation for group delay. Plot the effects in the time domain.
6. Plot the magnitude response of the combined three filters and the PSD of the final filtered ECG that was derived in the previous question. (refer to the slide ‘Overall filtering process applied on an ECG’).

4 IIR Filters

This section explores the realisation of IIR Butterworth filters, the effect of non-linear phase response, forward-backward filtering, and a comparison to the FIR filters implemented in the previous section. Use Python/SciPy functions to calculate IIR filter parameters directly.

4.1 Realizing IIR Filters

1. Obtain the filter coefficients of a Butterworth lowpass filter with the same cutoff frequency used to implement the FIR lowpass filter in the previous section, using `scipy.signal.butter()`.
2. Visualise the magnitude response, phase response and group delay using `scipy.signal.freqz()` and `scipy.signal.group_delay()`.

3. Obtain the filter coefficients of the highpass filter and the comb filter, and visualize them. For the comb filter use `scipy.signal.iircomb()`. Note that SciPy's `iircomb` is parameterised by notch frequency (w_0 , Hz) and quality factor (Q).
4. Plot the magnitude response of the combined three filters and compare it with the corresponding FIR filter plot from Section 3.2.

4.2 Filtering methods for IIR filters

1. Apply forward filtering by using `scipy.signal.lfilter()` to filter the `ECG_with_noise.npy` signal with the IIR filters realised in the previous section.
2. Apply forward-backward filtering by using `scipy.signal.filtfilt()` to filter the same signal with the IIR filters realised in the previous section.
3. Generate overlapping time domain plots of the FIR filtered ECG, IIR forward filtered ECG and IIR forward-backward filtered ECG. Compare the effect with respect to theoretical interpretations. (zoom-in to include only two ECG pulses to improve clarity)
4. Generate overlapping plots of the PSDs of the FIR-filtered ECG, IIR forward-filtered ECG, and IIR forward-backward-filtered ECG, and compare with respect to theoretical interpretations.